REFERENCE · CHAPTERS 7 & 8

Envoy Proxy in GCP Agent Governance: Managed Gateways vs. Cloud Run Sidecars

How Envoy's L7 filter chain (ext_authz, ext_proc, route allowlists) enforces out-of-process ingress and egress governance.

1. Why Out-of-Process Envoy Beats In-Process Python Middleware

Security checks implemented inside the Python agent container share fate with the agent: an omitted decorator, a prompt injection, or a compromised dependency can bypass them. **Envoy Proxy** runs out-of-process and evaluates every HTTP/gRPC call against a deterministic filter chain before forwarding traffic.

DEPLOYMENT PATTERN	WHEN TO USE IT	HOW ENVOY RUNS	KEY CONTROLS ENFORCED
Pattern A: Google-Managed Envoy (`Ingress/Egress Agent Gateway` & `ALB Service Extensions`)	Agents hosted on **Agent Runtime** or **Gemini Enterprise** (or behind External ALB).	Managed by Google Cloud; configured declaratively via IAM, IAP CEL policies, and Model Armor templates (zero Envoy YAML).	SPIFFE mTLS + DPoP, `Agent Registry` discovery, IAP `REQUEST_AUTHZ` (`mcp.toolName` , `mcp.tool.isReadOnly == true`), inline Model Armor (`CONTENT_AUTHZ`).
Pattern B: Self-Managed Cloud Run 3-Container Pod (`agent` + `envoy` + `tokenx`)	Agents hosted directly on **Cloud Run** or **GKE** that must perform OAuth 2.0 RFC 8693 OBO token exchange.	Envoy runs as a sidecar container on `127.0.0.1:15001` calling a local `tokenx` helper container on `127.0.0.1:9001` via `ext_authz` .	Zero OAuth `client_secret` in the agent container; agent never sees downstream token `Token_obo` ; hard outbound route allowlist (unknown hosts get `403`).

cloud_run_service.yaml — Multi-Container Cloud Run Deployment (agent + envoy + tokenx)

```yaml
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: governed-orders-agent
  annotations:
```

```yaml
    run.googleapis.com/ingress: internal-and-cloud-load-balancing
    run.googleapis.com/vpc-access-egress: all-traffic
spec:
  template:
    spec:
      serviceAccountName: orders-agent-sa@app-prod.iam.gserviceaccount
      containers:
      - name: agent
        image: us-central1-docker.pkg.dev/app-prod/agents/orders-agent
        ports: [{ containerPort: 8080 }]
        env:
        - name: MCP_ORDERS_URL
          value: "http://127.0.0.1:15001/orders-mcp/mcp" # Calls local
      - name: envoy-egress
        image: envoyproxy/envoy:v1.31-latest
        args: ["-c", "/etc/envoy/envoy.yaml"]
      - name: tokenx
        image: us-central1-docker.pkg.dev/app-prod/agents/tokenx@sha25
        env:
        - name: IDP_CLIENT_SECRET
          valueFrom:
            secretKeyRef: { name: idp-obo-client-secret, key: latest }
```